

Database Implementation

Proof of Connection

```
mysql> SHOW databases;
+-----+
| Database |
+-----+
| college_db |
| information_schema |
| mysql |
| performance_schema |
| sys |
+-----+
5 rows in set (0.077 sec)

mysql> USE college_db;
Database changed
mysql> SHOW tables;
+-----+
| Tables_in_college_db |
+-----+
| college_majors |
| colleges_dynamic |
| colleges_static |
| majors |
| regions |
| religions |
| settings |
| sizes |
| specprefs |
| states |
| user_accounts |
| user_majors |
| user_responses |
+-----+
13 rows in set (0.022 sec)

mysql> |
```

DDL Commands

```
CREATE TABLE colleges_static (
  college_id INT PRIMARY KEY,
  name VARCHAR(255) NOT NULL,
  website VARCHAR(255),
  city VARCHAR(255),
  state VARCHAR(255),
  year_established INT,
  type VARCHAR(255),
  religious_affiliation VARCHAR(255),
  online_only BOOLEAN,
  hbcu BOOLEAN,
```

```
    aapi BOOLEAN,  
    tribal BOOLEAN,  
    hispanic BOOLEAN,  
    men_only BOOLEAN,  
    women_only BOOLEAN  
);
```

```
CREATE TABLE colleges_dynamic (  
    college_id INT REFERENCES colleges_static(college_id),  
    year INT NOT NULL,  
    admission_rate FLOAT,  
    sat_rw_mid INT,  
    sat_math_mid INT,  
    sat_avg INT,  
    graduation_rate FLOAT,  
    median_earnings_10yr FLOAT,  
    avg_cost_of_attendance FLOAT,  
    in_state_tuition_fees FLOAT,  
    out_state_tuition_fees FLOAT,  
    net_price_0_30k FLOAT,  
    median_starting_debt FLOAT,  
    num_students INT,  
    PRIMARY KEY (college_id, year)  
);
```

```
CREATE TABLE majors (  
    major_id INT PRIMARY KEY,  
    major_name VARCHAR(255) NOT NULL,  
    code VARCHAR(255),  
    stem_flag BOOLEAN,  
    starting_salary REAL  
);
```

```
CREATE TABLE college_majors (  
    college_id INT REFERENCES colleges_static(college_id),  
    major_id INT REFERENCES majors(major_id),  
    PRIMARY KEY (college_id, major_id)  
);
```

```
CREATE TABLE user_responses (  
    id INT PRIMARY KEY,  
    relImp INT,  
    sizeImp INT,  
    allMajors BOOLEAN,  
    satMath INT,  
    satEng INT,  
    act INT,  
    settingImp INT,  
    regionImp INT,  
    stateImp INT,  
    income INT  
);  
  
CREATE TABLE user_accounts (  
    id INT PRIMARY KEY,  
    email VARCHAR(255),  
    password VARCHAR(255),  
    name VARCHAR(255),  
    CONSTRAINT fk_user_accounts_user_responses  
        FOREIGN KEY (id) REFERENCES user_responses(id)  
        ON DELETE CASCADE  
        ON UPDATE CASCADE  
);  
  
CREATE TABLE religions (  
    id INT PRIMARY KEY,  
    user_id INT NOT NULL,  
    code INT,  
    name VARCHAR(255),  
    CONSTRAINT fk_religions_user  
        FOREIGN KEY (user_id) REFERENCES user_responses(id)  
        ON DELETE CASCADE  
        ON UPDATE CASCADE  
);  
  
CREATE TABLE sizes (  
    id INT PRIMARY KEY,  
    user_id INT NOT NULL,
```

```
code INT,  
name VARCHAR(255),  
CONSTRAINT fk_sizes_user  
    FOREIGN KEY (user_id) REFERENCES user_responses(id)  
    ON DELETE CASCADE  
    ON UPDATE CASCADE  
);  
  
CREATE TABLE user_majors (  
    id INT PRIMARY KEY,  
    user_id INT NOT NULL,  
    code INT,  
    name VARCHAR(255),  
    CONSTRAINT fk_majors_user  
        FOREIGN KEY (user_id) REFERENCES user_responses(id)  
        ON DELETE CASCADE  
        ON UPDATE CASCADE  
);  
  
CREATE TABLE settings (  
    id INT PRIMARY KEY,  
    user_id INT NOT NULL,  
    code INT,  
    name VARCHAR(255),  
    CONSTRAINT fk_settings_user  
        FOREIGN KEY (user_id) REFERENCES user_responses(id)  
        ON DELETE CASCADE  
        ON UPDATE CASCADE  
);  
  
CREATE TABLE regions (  
    id INT PRIMARY KEY,  
    user_id INT NOT NULL,  
    code INT,  
    name VARCHAR(255),  
    CONSTRAINT fk_regions_user  
        FOREIGN KEY (user_id) REFERENCES user_responses(id)  
        ON DELETE CASCADE  
        ON UPDATE CASCADE
```

```
);

CREATE TABLE states (
  id INT PRIMARY KEY,
  user_id INT NOT NULL,
  code INT,
  name VARCHAR(255),
  CONSTRAINT fk_states_user
    FOREIGN KEY (user_id) REFERENCES user_responses(id)
    ON DELETE CASCADE
    ON UPDATE CASCADE
);

CREATE TABLE specPrefs (
  id INT PRIMARY KEY,
  user_id INT NOT NULL,
  code INT,
  name VARCHAR(255),
  CONSTRAINT fk_specPrefs_user
    FOREIGN KEY (user_id) REFERENCES user_responses(id)
    ON DELETE CASCADE
    ON UPDATE CASCADE
);
```

Table Sizes

```
mysql> SELECT COUNT(*) FROM colleges_dynamic;
+-----+
| COUNT(*) |
+-----+
|      2471 |
+-----+
1 row in set (0.018 sec)
```

```
mysql> SELECT COUNT(*) FROM colleges_static;
+-----+
| COUNT(*) |
+-----+
|      2471 |
+-----+
1 row in set (0.030 sec)
```

```
mysql> SELECT COUNT(*) FROM college_majors;
+-----+
| COUNT(*) |
+-----+
|      31828 |
+-----+
1 row in set (0.046 sec)
```

```
mysql> SELECT COUNT(*) FROM majors;
+-----+
| COUNT(*) |
+-----+
|        38 |
+-----+
1 row in set (0.028 sec)
```

```
mysql> SELECT COUNT(*) FROM user_responses;
+-----+
| COUNT(*) |
+-----+
|      1000 |
+-----+
1 row in set (0.032 sec)
```

```
mysql> SELECT COUNT(*) FROM user_accounts;
+-----+
| COUNT(*) |
+-----+
|       900 |
+-----+
1 row in set (0.033 sec)
```

```
mysql> SELECT COUNT(*) FROM religions;
+-----+
| COUNT(*) |
+-----+
|      1017 |
+-----+
1 row in set (0.019 sec)
```

```
mysql> SELECT COUNT(*) FROM sizes;
+-----+
| COUNT(*) |
+-----+
|      1500 |
+-----+
1 row in set (0.021 sec)
```

```
mysql> SELECT COUNT(*) FROM settings;
+-----+
| COUNT(*) |
+-----+
|      1021 |
+-----+
1 row in set (0.017 sec)
```

```
mysql> SELECT COUNT(*) FROM user_majors;
+-----+
| COUNT(*) |
+-----+
|      2538 |
+-----+
1 row in set (0.021 sec)
```

```
mysql> SELECT COUNT(*) FROM regions;
+-----+
| COUNT(*) |
+-----+
|      1477 |
+-----+
1 row in set (0.020 sec)
```

```
mysql> SELECT COUNT(*) FROM states;
+-----+
| COUNT(*) |
+-----+
|      2043 |
+-----+
1 row in set (0.023 sec)
```

```
mysql> SELECT COUNT(*) FROM specprefs;
+-----+
| COUNT(*) |
+-----+
|      509 |
+-----+
1 row in set (0.014 sec)
```

Advanced Queries

Query 1

Purpose

Recommend colleges for a specific user based on selected majors, SAT score, and affordability.

Select colleges that offer the user's majors and, according to the most recent data available, have an average SAT score lower than that of the user's (within 50 points) and cost of attendance below 35% of the user's household income.

Query

```
SELECT
  cs.college_id,
  cs.name AS college_name,
  cs.state,
  cd.year,
  cd.sat_avg,
  cd.avg_cost_of_attendance,
  COUNT(DISTINCT um.code) AS matching_majors
FROM user_responses ur
JOIN user_majors um
```

```

    ON ur.id = um.user_id
JOIN majors m
    ON um.name = m.major_name
JOIN college_majors cm
    ON m.major_id = cm.major_id
JOIN colleges_static cs
    ON cm.college_id = cs.college_id
JOIN colleges_dynamic cd
    ON cs.college_id = cd.college_id
WHERE ur.id = 1
    AND cd.year = (
        SELECT MAX(cd2.year)
        FROM colleges_dynamic cd2
        WHERE cd2.college_id = cd.college_id
    )
    AND (
        ur.satMath IS NULL
        OR ur.satEng IS NULL
        OR cd.sat_avg <= (ur.satMath + ur.satEng + 50)
    )
    AND (
        ur.income IS NULL
        OR cd.avg_cost_of_attendance <= ur.income * 0.35
    )
GROUP BY
    cs.college_id,
    cs.name,
    cs.state,
    cd.year,
    cd.sat_avg,
    cd.avg_cost_of_attendance
HAVING COUNT(DISTINCT um.code) > 0
ORDER BY matching_majors DESC, cd.avg_cost_of_attendance ASC
LIMIT 15;

```


Output

```
--> WHERE cd2.college_id = cd.college_id
--> )
--> AND (
-->   ur.satMath IS NULL
-->   OR ur.satEng IS NULL
-->   OR cd.sat_avg <= (ur.satMath + ur.satEng + 50)
--> )
--> AND (
-->   ur.income IS NULL
-->   OR cd.avg_cost_of_attendance <= ur.income * 0.35
--> )
--> GROUP BY
-->   cs.college_id,
-->   cs.name,
-->   cs.state,
-->   cd.year,
-->   cd.sat_avg,
-->   cd.avg_cost_of_attendance
--> HAVING COUNT(DISTINCT um.code) > 0
--> ORDER BY matching_majors DESC, cd.avg_cost_of_attendance ASC
--> LIMIT 15;
```

college_id	college_name	state	year	sat_avg	avg_cost_of_attendance	matching_majors
119137	Moorpark College	CA	2023	NULL	8015	2
409315	South Texas College	TX	2023	NULL	9561	2
224615	Dallas College	TX	2023	NULL	9704	2
241377	Caribbean University-Bayamon	PR	2023	NULL	9835	2
203748	Lorain County Community College	OH	2023	NULL	9860	2
205470	Sinclair Community College	OH	2023	NULL	11003	2
122977	Santa Monica College	CA	2023	NULL	11162	2
118976	Modesto Junior College	CA	2023	NULL	11487	2
170240	Henry Ford College	MI	2023	NULL	11784	2
225423	Houston Community College	TX	2023	NULL	12009	2
133386	Daytona State College	FL	2023	NULL	12050	2
121619	Santa Ana College	CA	2023	NULL	12375	2
230597	Snow College	UT	2023	NULL	12514	2
142559	College of Southern Idaho	ID	2023	NULL	12803	2
224350	Del Mar College	TX	2023	NULL	12895	2

15 rows in set (0.178 sec)

Query 2

Purpose

This query retrieves colleges in the user's preferred regions that offer the user's selected majors, ranking them by how many of those majors each college provides. The query assumes the user actually has preferred regions.

Query

```
SELECT
    cs.college_id,
    cs.name,
    cs.city,
    cs.state,
    COUNT(DISTINCT um.code) AS matched_major_count
FROM colleges_static cs
JOIN college_majors cm
    ON cs.college_id = cm.college_id
JOIN majors m
    ON cm.major_id = m.major_id
JOIN user_majors um
    ON m.major_id = um.code
WHERE um.user_id = 4
    AND (
        CASE
            WHEN cs.state IN
                ('CT', 'ME', 'MA', 'NH', 'RI', 'VT', 'NJ', 'NY', 'PA') THEN 'Northeast'
            WHEN cs.state IN
                ('IL', 'IN', 'MI', 'OH', 'WI', 'IA', 'KS', 'MN', 'MO', 'NE', 'ND', 'SD') THEN
                'Midwest'
            WHEN cs.state IN
                ('DE', 'FL', 'GA', 'MD', 'NC', 'SC', 'VA', 'DC', 'WV', 'AL', 'KY', 'MS', 'TN', 'AR',
                'LA', 'OK', 'TX') THEN 'South'
            WHEN cs.state IN
                ('AZ', 'CO', 'ID', 'MT', 'NV', 'NM', 'UT', 'WY', 'AK', 'CA', 'HI', 'OR', 'WA')
            THEN 'West'
        END
    ) IN (
        SELECT r.name
        FROM regions r
        WHERE r.user_id = 4
    )
GROUP BY cs.college_id, cs.name, cs.city, cs.state
```

```
HAVING COUNT(DISTINCT um.code) >= 1
ORDER BY matched_major_count DESC, cs.name
LIMIT 15;
```

Output

```
-> COUNT(DISTINCT um.code) AS matched_major_count
-> FROM colleges_static cs
-> JOIN college_majors cm
->   ON cs.college_id = cm.college_id
-> JOIN majors m
->   ON cm.major_id = m.major_id
-> JOIN user_majors um
->   ON m.major_id = um.code
-> WHERE um.user_id = 4
->   AND (
->     CASE
->       WHEN cs.state IN ('CT', 'ME', 'MA', 'NH', 'RI', 'VT', 'NJ', 'NY', 'PA') THEN 'Northeast'
->       WHEN cs.state IN ('IL', 'IN', 'MI', 'OH', 'WI', 'IA', 'KS', 'MN', 'MO', 'NE', 'ND', 'SD') THEN 'Midwest'
->       WHEN cs.state IN ('DE', 'FL', 'GA', 'MD', 'NC', 'SC', 'VA', 'DC', 'WV', 'AL', 'KY', 'MS', 'TN', 'AR', 'LA', 'OK', 'TX') THEN 'South'
->       WHEN cs.state IN ('AZ', 'CO', 'ID', 'MT', 'NV', 'NM', 'UT', 'WY', 'AK', 'CA', 'HI', 'OR', 'WA') THEN 'West'
->     END
->   ) IN (
->     SELECT r.name
->     FROM regions r
->     WHERE r.user_id = 4
->   )
-> GROUP BY cs.college_id, cs.name, cs.city, cs.state
-> HAVING COUNT(DISTINCT um.code) >= 1
-> ORDER BY matched_major_count DESC, cs.name
-> LIMIT 15;
```

college_id	name	city	state	matched_major_count
222178	Abilene Christian University	Abilene	TX	2
138558	Abraham Baldwin Agricultural College	Tifton	GA	2
100654	Alabama A & M University	Normal	AL	2
100724	Alabama State University	Montgomery	AL	2
138716	Albany State University	Albany	GA	2
175342	Alcorn State University	Alcorn State	MS	2
156189	Alice Lloyd College	Pippa Passes	KY	2
449339	American Public University System	Charles Town	WV	2
131159	American University	Washington	DC	2
217633	Anderson University	Anderson	SC	2
138761	Andrew College	Cuthbert	GA	2
222831	Angelo State University	San Angelo	TX	2
197869	Appalachian State University	Boone	NC	2
106458	Arkansas State University	Jonesboro	AR	2
106467	Arkansas Tech University	Russellville	AR	2

15 rows in set (0.126 sec)

Query 3

Purpose

This query identifies colleges that offer the user's selected majors and ranks them based on a value metric defined as the ratio of median earnings 10 years after enrollment to the average cost

of attendance. It helps users find colleges that maximize return on investment while still matching their academic interests.

Query

```
SELECT
  cs.college_id,
  cs.name AS college_name,
  cs.state,
  cd.year,
  AVG(cd.median_earnings_10yr) AS avg_earnings,
  AVG(cd.avg_cost_of_attendance) AS avg_cost,
  (AVG(cd.median_earnings_10yr) /
  NULLIF(AVG(cd.avg_cost_of_attendance), 0)) AS value_ratio,
  COUNT(DISTINCT cm.major_id) AS matched_majors
FROM colleges_static cs
JOIN colleges_dynamic cd
  ON cs.college_id = cd.college_id
JOIN college_majors cm
  ON cs.college_id = cm.college_id
JOIN majors m
  ON cm.major_id = m.major_id
JOIN user_majors um
  ON m.major_id = um.code
WHERE um.user_id = 903
  AND cd.year = (
    SELECT MAX(cd2.year)
    FROM colleges_dynamic cd2
    WHERE cd2.college_id = cd.college_id
  )
GROUP BY cs.college_id, cs.name, cs.state, cd.year
HAVING COUNT(DISTINCT cm.major_id) >= 1
ORDER BY value_ratio DESC, avg_cost ASC
LIMIT 15;
```

Output

```
--> cs.state,
--> cd.year,
--> AVG(cd.median_earnings_10yr) AS avg_earnings,
--> AVG(cd.avg_cost_of_attendance) AS avg_cost,
--> (AVG(cd.median_earnings_10yr) / NULLIF(AVG(cd.avg_cost_of_attendance), 0)) AS value_ratio,
--> COUNT(DISTINCT cm.major_id) AS matched_majors
--> FROM colleges_static cs
--> JOIN colleges_dynamic cd
-->   ON cs.college_id = cd.college_id
--> JOIN college_majors cm
-->   ON cs.college_id = cm.college_id
--> JOIN majors m
-->   ON cm.major_id = m.major_id
--> JOIN user_majors um
-->   ON m.major_id = um.code
--> WHERE um.user_id = 903
--> AND cd.year = (
-->   SELECT MAX(cd2.year)
-->   FROM colleges_dynamic cd2
-->   WHERE cd2.college_id = cd.college_id
--> )
--> GROUP BY cs.college_id, cs.name, cs.state, cd.year
--> HAVING COUNT(DISTINCT cm.major_id) >= 1
--> ORDER BY value_ratio DESC, avg_cost ASC
--> LIMIT 15;
```

college_id	college_name	state	year	avg_earnings	avg_cost	value_ratio	matched_majors
121619	Santa Ana College	CA	2023	43552	12375	3.5193535353535355	1
142559	College of Southern Idaho	ID	2023	40916	12803	3.1958134812153403	1
181394	University of Nebraska at Omaha	NE	2023	53909	21120	2.55250946969697	1
236708	Spokane Falls Community College	WA	2023	38955	15685	2.4835830411220914	1
240453	University of Wisconsin-Milwaukee	WI	2023	54990	22756	2.4165055370012305	1
114789	Fresno City College	CA	2023	37361	17198	2.172403767879986	1
161217	University of Maine at Augusta	ME	2023	40342	19052	2.1174679823640563	1
207670	Rose State College	OK	2023	37555	18655	2.0131332082551596	1
180054	Blackfeet Community College	MT	2023	22953	11405	2.0125383603682594	1
130493	Southern Connecticut State University	CT	2023	55043	29673	1.854986014221683	1
213349	Kutztown University of Pennsylvania	PA	2023	53775	29866	1.800542422821938	1
127884	Pueblo Community College	CO	2023	37434	22002	1.701390782656122	1
176372	University of Southern Mississippi	MS	2023	44140	26566	1.6615222464804638	1
131283	The Catholic University of America	DC	2023	73250	75481	0.9704428929134484	1
154022	Ashford University	CA	2023	35404	37594	0.9417460233015906	1

15 rows in set (0.074 sec)

Indexing Analysis

Query 1

Baseline performance

```
| -> Limit: 15 row(s) (actual time=46..46 rows=15 loops=1)
      -> Sort: matching_majors DESC, cd.avg_cost_of_attendance (actual time=46..46 rows=15 loops=1)
            -> Filter: ('count(distinct user_majors.'code')' > 0) (actual time=42.6..45.3 rows=1041 loops=1)
            -> Stream results (actual time=42.6..45.2 rows=1041 loops=1)
                  -> Group aggregate: count(distinct user_majors.'code'), count(distinct user_majors.'code') (actual time=42.6..44.3 rows=1041 loops=1)
                        -> Sort: cs.college_id, cs.'name', cs.state, cd.'year', cd.sat_avg, cd.avg_cost_of_attendance (actual time=42.6..42.8 rows=1512 loops=1)
                                -> Stream results (cost=5830 rows=9140) (actual time=0.371..40 rows=1512 loops=1)
                                        -> Nested loop inner join (cost=5830 rows=9140) (actual time=0.364..38.1 rows=1512 loops=1)
                                                -> Nested loop inner join (cost=2631 rows=9140) (actual time=0.295..10.4 rows=2076 loops=1)
                                                        -> Nested loop inner join (cost=955 rows=9140) (actual time=0.281..1.52 rows=2076 loops=1)
                                                                -> Filter: (m.major_name = um.'name') (cost=12.7 rows=11.4) (actual time=0.164..0.218 rows=2 loops=1)
                                                                      -> Inner hash join (<hash>(m.major_name)=<hash>(um.'name')) (cost=12.7 rows=11.4) (actual time=0.161..0.212 rows=2 loops=1)
                                                                              -> Table scan on m (cost=0.211 rows=38) (actual time=0.0197..0.0549 rows=38 loops=1)
                                                                                    -> Hash
                                                                                          -> Index lookup on um using idx_user_majors_user_id_code (user_id = 1) (cost=1.05 rows=3) (actual time=0.0863..0.0918 rows=3 loops=1)
                                                                                                -> Covering index lookup on cm using idx_college_majors (major_id = m.major_id) (cost=9.52 rows=802) (actual time=0.1..0.539 rows=1038 loops=2)
                                                                                                      -> Single-row index lookup on cs using PRIMARY (college_id = cm.college_id) (cost=0.0833 rows=1) (actual time=0.0039..0.00396 rows=1 loops=2076)
                                                                                                            -> Filter: ((cd.'year' = (select #2)) and (<cache>((NULL is null)) or <cache>((NULL is null)) or (cd.sat_avg <= <cache>(((NULL + NULL) + 50)))) and (<cache>(('150000' is null)) or (cd.avg_cost_of_attendance <= <cache>(('150000' * 0.35)))) (cost=0.25 rows=1) (actual time=0.012..0.0131 rows=0.728 loops=2076)
                                                                                                                  -> Index lookup on cd using PRIMARY (college_id = cm.college_id) (cost=0.25 rows=1) (actual time=0.00364..0.0049 rows=1 loops=2076)
                                                                                                                        -> Select #2 (subquery in condition; dependent)
                                                                                                                              -> Aggregate: max(cd2.'year') (cost=0.58 rows=1) (actual time=0.00621..0.00627 rows=1 loops=2076)
                                                                                                                                    -> Covering index lookup on cd2 using PRIMARY (college_id = cd.college_id) (cost=0.35 rows=1) (actual time=0.00454..0.00565 rows=1 loops=2076)
|
```

Index 1 (major names)

```
CREATE INDEX idx_majors_name ON majors(major_name);
```

Output:

```
| -> Limit: 15 row(s) (actual time=54.9..54.9 rows=15 loops=1)
      -> Sort: matching_majors DESC, cd.avg_cost_of_attendance (actual time=54.9..54.9 rows=15 loops=1)
            -> Filter: ('count(distinct user_majors.'code')' > 0) (actual time=50..53.9 rows=1041 loops=1)
            -> Stream results (actual time=50..53.8 rows=1041 loops=1)
                  -> Group aggregate: count(distinct user_majors.'code'), count(distinct user_majors.'code') (actual time=49.9..52.5 rows=1041 loops=1)
                        -> Sort: cs.college_id, cs.'name', cs.state, cd.'year', cd.sat_avg, cd.avg_cost_of_attendance (actual time=49.9..50.3 rows=1512 loops=1)
                                -> Stream results (cost=1928 rows=2405) (actual time=0.247..47 rows=1512 loops=1)
                                        -> Nested loop inner join (cost=1928 rows=2405) (actual time=0.243..45.1 rows=1512 loops=1)
                                                -> Nested loop inner join (cost=1086 rows=2405) (actual time=0.194..12.5 rows=2076 loops=1)
                                                        -> Nested loop inner join (cost=244 rows=2405) (actual time=0.178..1.73 rows=2076 loops=1)
                                                                -> Filter: (um.'name' is not null) (cost=1.05 rows=3) (actual time=0.0436..0.0548 rows=3 loops=1)
                                                                      -> Index lookup on um using fk_majors_user (user_id = 1) (cost=1.05 rows=3) (actual time=0.0423..0.0523 rows=3 loops=1)
                                                                              -> Covering index lookup on m using idx_majors_name (major_name = um.'name') (cost=0.283 rows=1) (actual time=0.0157..0.022 rows=0.667 loops=3)
                                                                                                -> Covering index lookup on cm using major_id (major_id = m.major_id) (cost=27.3 rows=802) (actual time=0.106..0.653 rows=1038 loops=2)
                                                                                                      -> Single-row index lookup on cs using PRIMARY (college_id = cm.college_id) (cost=0.25 rows=1) (actual time=0.0048..0.00486 rows=1 loops=2076)
                                                                                                            -> Filter: ((cd.'year' = (select #2)) and (<cache>((NULL is null)) or <cache>((NULL is null)) or (cd.sat_avg <= <cache>(((NULL + NULL) + 50)))) and (<cache>(('150000' is null)) or (cd.avg_cost_of_attendance <= <cache>(('150000' * 0.35)))) (cost=0.25 rows=1) (actual time=0.0141..0.0153 rows=0.728 loops=2076)
                                                                                                                  -> Index lookup on cd using PRIMARY (college_id = cm.college_id) (cost=0.25 rows=1) (actual time=0.00432..0.0058 rows=1 loops=2076)
                                                                                                                        -> Select #2 (subquery in condition; dependent)
                                                                                                                              -> Aggregate: max(cd2.'year') (cost=0.58 rows=1) (actual time=0.00725..0.00732 rows=1 loops=2076)
                                                                                                                                    -> Covering index lookup on cd2 using PRIMARY (college_id = cd.college_id) (cost=0.35 rows=1) (actual time=0.00529..0.0066 rows=1 loops=2076)
|
```

Index 2 (SAT scores)

```
CREATE INDEX idx_sat ON colleges_dynamic(sat_avg);
```

Output:

```
| -> Limit: 15 row(s) (actual time=54.2..54.2 rows=15 loops=1)
    -> Sort: matching_majors DESC, cd.avg_cost_of_attendance (actual time=54.2..54.2 rows=15 loops=1)
    -> Filter: ('count(distinct user_majors.'code')' > 0) (actual time=49.7..53.4 rows=1041 loops=1)
    -> Stream results (actual time=49.7..53.2 rows=1041 loops=1)
    -> Group aggregate: count(distinct user_majors.'code'), count(distinct user_majors.'code') (actual time=49.7..52.1 rows=1041 loops=1)
    -> Sort: cs.college_id, cs.name, cs.state, cd.year, cd.sat_avg, cd.avg_cost_of_attendance (actual time=49.7..50 rows=1512 loops=1)
    -> Stream results (cost=1928 rows=2405) (actual time=0.215..45 rows=1512 loops=1)
    -> Nested loop inner join (cost=1928 rows=2405) (actual time=0.215..45 rows=1512 loops=1)
    -> Nested loop inner join (cost=1086 rows=2405) (actual time=0.163..12.3 rows=2076 loops=1)
    -> Nested loop inner join (cost=244 rows=2405) (actual time=0.15..1.61 rows=2076 loops=1)
    -> Nested loop inner join (cost=2.1 rows=3) (actual time=0.0541..0.122 rows=2 loops=1)
    -> Filter: (um.name is not null) (cost=1.05 rows=3) (actual time=0.0356..0.0454 rows=3 loops=1)
    -> Index lookup on um using fk_majors_user (user_id = 1) (cost=1.05 rows=3) (actual time=0.0344..0.0429 rows=3 loops=1)
    -> Covering index lookup on m using idx_majors_name (major_name = um.name) (cost=0.283 rows=1) (actual time=0.0197..0.025 r
ows=0.667 loops=3)
    -> Covering index lookup on cm using major_id (major_id = m.major_id) (cost=27.3 rows=802) (actual time=0.13..0.623 rows=1038 loo
ps=2)
    -> Single-row index lookup on cs using PRIMARY (college_id = cm.college_id) (cost=0.25 rows=1) (actual time=0.00479..0.00485 rows=1 l
oops=2076)
    -> Filter: ((cd.year = (select #2)) and (<cache>((NULL is null)) or <cache>((NULL is null)) or (cd.sat_avg <= <cache>((NULL + NULL) + 5
0)))) and (<cache>('150000' is null)) or (cd.avg_cost_of_attendance <= <cache>('150000' * 0.35)))) (cost=0.25 rows=1) (actual time=0.0142..0.0154 rows=0.728 loops=207
6)
    -> Index lookup on cd using PRIMARY (college_id = cm.college_id) (cost=0.25 rows=1) (actual time=0.00438..0.00592 rows=1 loops=2076)
    -> Select #2 (subquery in condition; dependent)
    -> Aggregate: max(cd2.year) (cost=0.58 rows=1) (actual time=0.00727..0.00732 rows=1 loops=2076)
    -> Covering index lookup on cd2 using PRIMARY (college_id = cd.college_id) (cost=0.35 rows=1) (actual time=0.00533..0.00664 r
ows=1 loops=2076)
|
```

Index 3 (SELECT attributes)

```
CREATE INDEX idx_cd_cover ON colleges_dynamic(college_id, year,
sat_avg, avg_cost_of_attendance);
```

Output:

```
| -> Limit: 15 row(s) (actual time=44.44 rows=15 loops=1)
    -> Sort: matching_majors DESC, cd.avg_cost_of_attendance (actual time=44.44 rows=15 loops=1)
    -> Filter: ('count(distinct user_majors.'code')' > 0) (actual time=39.8..43.3 rows=1041 loops=1)
    -> Stream results (actual time=39.8..43.1 rows=1041 loops=1)
    -> Group aggregate: count(distinct user_majors.'code'), count(distinct user_majors.'code') (actual time=39.8..42 rows=1041 loops=1)
    -> Sort: cs.college_id, cs.name, cs.state, cd.year, cd.sat_avg, cd.avg_cost_of_attendance (actual time=39.8..40.1 rows=1512 loops=1)
    -> Stream results (cost=1928 rows=2405) (actual time=0.215..37.6 rows=1512 loops=1)
    -> Nested loop inner join (cost=1928 rows=2405) (actual time=0.211..36 rows=1512 loops=1)
    -> Nested loop inner join (cost=1086 rows=2405) (actual time=0.162..9.79 rows=2076 loops=1)
    -> Nested loop inner join (cost=244 rows=2405) (actual time=0.149..1.25 rows=2076 loops=1)
    -> Nested loop inner join (cost=2.1 rows=3) (actual time=0.0547..0.0892 rows=2 loops=1)
    -> Filter: (um.name is not null) (cost=1.05 rows=3) (actual time=0.0358..0.0437 rows=3 loops=1)
    -> Index lookup on um using fk_majors_user (user_id = 1) (cost=1.05 rows=3) (actual time=0.0347..0.0413 rows=3 loops=1)
    -> Covering index lookup on m using idx_majors_name (major_name = um.name) (cost=0.283 rows=1) (actual time=0.0099..0.0145
rows=0.667 loops=3)
    -> Covering index lookup on cm using major_id (major_id = m.major_id) (cost=27.3 rows=802) (actual time=0.0894..0.47 rows=1038 lo
ops=2)
    -> Single-row index lookup on cs using PRIMARY (college_id = cm.college_id) (cost=0.25 rows=1) (actual time=0.00376..0.00382 rows=1 l
oops=2076)
    -> Filter: ((cd.year = (select #2)) and (<cache>((NULL is null)) or <cache>((NULL is null)) or (cd.sat_avg <= <cache>((NULL + NULL) + 5
0)))) and (<cache>('150000' is null)) or (cd.avg_cost_of_attendance <= <cache>('150000' * 0.35)))) (cost=0.25 rows=1) (actual time=0.0113..0.0123 rows=0.728 loops=207
6)
    -> Index lookup on cd using PRIMARY (college_id = cm.college_id) (cost=0.25 rows=1) (actual time=0.00341..0.00462 rows=1 loops=2076)
    -> Select #2 (subquery in condition; dependent)
    -> Aggregate: max(cd2.year) (cost=0.58 rows=1) (actual time=0.00584..0.0059 rows=1 loops=2076)
    -> Covering index lookup on cd2 using PRIMARY (college_id = cd.college_id) (cost=0.35 rows=1) (actual time=0.00424..0.00531 r
ows=1 loops=2076)
|
```

Analysis

We explored several indexing strategies targeting the most expensive parts of the query, focusing on join attributes and the correlated subquery. The most impactful change came from adding an

index on majors(major_name), which reduced the estimated cost from 5808 to 1928. This significant improvement occurred because the original execution plan performed a full table scan on the majors table during the join with user_majors, and the new index allowed MySQL to switch to an index lookup, dramatically reducing the number of rows examined early in the query. The filter cost went down from 12.7 to 1.05. Since this join appears near the beginning of the execution plan, optimizing it had a cascading effect on the rest of the query by shrinking intermediate results.

We also experimented with a subsequent index on colleges_dynamic(sat_avg) to optimize a subquery within WHERE and a broader covering index including filtering attributes. However, these indexes did not produce measurable improvements in cost. This is likely because MySQL was already using the primary key indices, and the dataset size or distribution did not make the additional year component beneficial for the MAX aggregation or other parts of the query. Additionally, the presence of OR conditions in the WHERE clause limited the optimizer's ability to leverage indexes for filtering, resulting in similar execution strategies before and after indexing. Overall, the experiment highlights that indexing join attributes, especially those causing full table scans, can yield substantial performance gains, while more complex or theoretically optimal indexes may have little effect depending on query structure and optimizer behavior.

In conclusion, the index on major names in the majors table is the most effective to use, since it removes the table scan during the join and results in the biggest gain in the efficiency of the query.

Query 2

Baseline performance

```
| -> Limit: 15 row(s) (actual time=33.1..33.1 rows=15 loops=1)
    -> Sort: matched_major_count DESC, cs.'name' (actual time=33.1..33.1 rows=15 loops=1)
        -> Filter: ('count(distinct user_majors.'code') >= 1) (actual time=29.5..32.3 rows=557 loops=1)
            -> Stream results (actual time=29.5..32.2 rows=557 loops=1)
                -> Group aggregate: count(distinct user_majors.'code'), count(distinct user_majors.'code') (actual time=29.5..31.6 rows=557 loops=1)
                    -> Sort: cs.college_id, cs.'name', cs.city, cs.state (actual time=29.4..29.7 rows=1415 loops=1)
                        -> Stream results (cost=1086 rows=2405) (actual time=0.304..27.1 rows=1415 loops=1)
                            -> Remove duplicate (um, cm) rows using temporary table (weedout) (cost=1086 rows=2405) (actual time=0.299..25.7 rows=1415 loops=1)
                                -> Nested loop inner join (cost=1086 rows=2405) (actual time=0.281..23.5 rows=1415 loops=1)
                                    -> Nested loop inner join (cost=245 rows=2405) (actual time=0.261..2.84 rows=4099 loops=1)
                                        -> Nested loop inner join (cost=2.45 rows=3) (actual time=0.111..0.127 rows=3 loops=1)
                                            -> Nested loop inner join (cost=1.4 rows=3) (actual time=0.0986..0.106 rows=3 loops=1)
                                                -> Index lookup on r using fk_regions_user (user_id = 4) (cost=0.35 rows=1) (actual time=0.0784..0.0814 rows=1 loops=1)
                                                    -> Filter: (um.'code' is not null) (cost=1.05 rows=3) (actual time=0.0188..0.0231 rows=3 loops=1)
                                                        -> Index lookup on um using fk_majors_user (user_id = 4) (cost=1.05 rows=3) (actual time=0.0176..0.0209 rows=3 loops=1)
1)
    -> Single-row covering index lookup on m using PRIMARY (major_id = um.'code') (cost=0.283 rows=1) (actual time=0.0061..0.0061 rows=1 loops=3)
7 rows=1 loops=3)
    -> Covering index lookup on cm using major_id (major_id = um.'code') (cost=27.3 rows=802) (actual time=0.127..0.755 rows=1366 loops=3)
    -> Filter: ((case when (cs.state in ('CT','ME','MA','NH','RI','VT','NJ','NY','PA')) then 'Northeast' when (cs.state in ('IL','IN','MI','OH','WI','IA','KS','MN','MO','NE','ND','SD')) then 'Midwest' when (cs.state in ('DE','FL','GA','MD','NC','SC','VA','DC','WV','AL','KY','MS','TN','AR','LA','OK','TX')) then 'South' when (cs.state in ('AZ','CO','ID','MT','NV','NM','UT','WY','AK','CA','HI','OR','WA')) then 'West' end) = r.'name') (cost=0.25 rows=1) (actual time=0.00478..0.00483 rows=0.345 loops=4099)
    -> Single-row index lookup on cs using PRIMARY (college_id = cm.college_id) (cost=0.25 rows=1) (actual time=0.00351..0.00356 rows=1 loops=4099)
1
|
```

Index 1 (user_major codes and user ID's)

```
CREATE INDEX idx_user_majors_user_id_code ON user_majors(user_id, code);
```

Output:

```
| -> Limit: 15 row(s) (actual time=32.3..32.3 rows=15 loops=1)
    -> Sort: matched_major_count DESC, cs.'name' (actual time=32.3..32.3 rows=15 loops=1)
        -> Filter: ('count(distinct user_majors.'code') >= 1) (actual time=29.6..31.6 rows=557 loops=1)
            -> Stream results (actual time=29.6..31.6 rows=557 loops=1)
                -> Group aggregate: count(distinct user_majors.'code'), count(distinct user_majors.'code') (actual time=29.6..29.8 rows=1415 loops=1)
                    -> Sort: cs.college_id, cs.'name', cs.city, cs.state (actual time=29.6..29.8 rows=1415 loops=1)
                        -> Stream results (cost=1086 rows=2405) (actual time=0.204..27.4 rows=1415 loops=1)
                            -> Remove duplicate (um, cm) rows using temporary table (weedout) (cost=1086 rows=2405) (actual time=0.199..25.9 rows=1415 loops=1)
                                -> Nested loop inner join (cost=1086 rows=2405) (actual time=0.193..24.4 rows=1415 loops=1)
                                    -> Nested loop inner join (cost=245 rows=2405) (actual time=0.169..2.7 rows=4099 loops=1)
                                        -> Nested loop inner join (cost=2.45 rows=3) (actual time=0.0906..0.0893 rows=3 loops=1)
                                            -> Nested loop inner join (cost=1.4 rows=3) (actual time=0.0584..0.0685 rows=3 loops=1)
                                                -> Index lookup on r using fk_regions_user (user_id = 4) (cost=0.35 rows=1) (actual time=0.0435..0.0476 rows=1 loops=1)
                                                    -> Filter: (um.'code' is not null) (cost=1.05 rows=3) (actual time=0.0133..0.0183 rows=3 loops=1)
                                                        -> Covering index lookup on um using idx_user_majors_user_id_code (user_id = 4) (cost=1.05 rows=3) (actual time=0.0124..0.0169 rows=3 loops=1)
    -> Single-row covering index lookup on m using PRIMARY (major_id = um.'code') (cost=0.283 rows=1) (actual time=0.00633..0.0064 rows=1 loops=3)
    -> Covering index lookup on cm using major_id (major_id = um.'code') (cost=27.3 rows=802) (actual time=0.0903..0.709 rows=1366 loops=3)
    -> Filter: ((case when (cs.state in ('CT','ME','MA','NH','RI','VT','NJ','NY','PA')) then 'Northeast' when (cs.state in ('IL','IN','MI','OH','WI','IA','KS','MN','MO','NE','ND','SD')) then 'Midwest' when (cs.state in ('DE','FL','GA','MD','NC','SC','VA','DC','WV','AL','KY','MS','TN','AR','LA','OK','TX')) then 'South' when (cs.state in ('AZ','CO','ID','MT','NV','NM','UT','WY','AK','CA','HI','OR','WA')) then 'West' end) = r.'name') (cost=0.25 rows=1) (actual time=0.00501..0.00507 rows=0.345 loops=4099)
    -> Single-row index lookup on cs using PRIMARY (college_id = cm.college_id) (cost=0.25 rows=1) (actual time=0.0037..0.00375 rows=1 loops=4099)
1
|
```

Index 2 (regions)

```
CREATE INDEX idx_regions ON regions(user_id, name);
```

Output:

```

| -> Limit: 15 row(s) (actual time=32.3..32.3 rows=15 loops=1)
    -> Sort: matched_major_count DESC, cs.'name' (actual time=32.3..32.3 rows=15 loops=1)
        -> Filter: ('count(distinct user_majors.'code') >= 1) (actual time=29.6..31.6 rows=557 loops=1)
        -> Stream results (actual time=29.6..31.6 rows=557 loops=1)
            -> Group aggregate: count(distinct user_majors.'code'), count(distinct user_majors.'code') (actual time=29.6..31.1 rows=557 loops=1)
            -> Sort: cs.college_id, cs.'name', cs.city, cs.state (actual time=29.6..29.8 rows=1415 loops=1)
            -> Stream results (cost=1086 rows=2405) (actual time=0.204..27.4 rows=1415 loops=1)
                -> Remove duplicate (um, cm) rows using temporary table (weedout) (cost=1086 rows=2405) (actual time=0.199..25.9 rows=1415 loops=1)
                -> Nested loop inner join (cost=1086 rows=2405) (actual time=0.193..24.4 rows=1415 loops=1)
                    -> Nested loop inner join (cost=245 rows=2405) (actual time=0.169..2.7 rows=4099 loops=1)
                        -> Nested loop inner join (cost=2.45 rows=3) (actual time=0.0706..0.0893 rows=3 loops=1)
                            -> Nested loop inner join (cost=1.4 rows=3) (actual time=0.0584..0.0685 rows=3 loops=1)
                                -> Index lookup on r using fk_regions_user (user_id = 4) (cost=0.35 rows=1) (actual time=0.0435..0.0476 rows=1 loops=1)
                                    -> Filter: (um.'code' is not null) (cost=1.05 rows=3) (actual time=0.0133..0.0183 rows=3 loops=1)
                                        -> Covering index lookup on um using idx_user_majors_user_id_code (user_id = 4) (cost=1.05 rows=3) (actual time=0.012
4..0.0169 rows=3 loops=1)
                                        -> Single-row covering index lookup on m using PRIMARY (major_id = um.'code') (cost=0.283 rows=1) (actual time=0.00633..0.006
4 rows=1 loops=1)
                                        -> Covering index lookup on cm using major_id (major_id = um.'code') (cost=27.3 rows=802) (actual time=0.0903..0.709 rows=1366 lo
ops=3)
                                -> Filter: ((case when (cs.state in ('CT','ME','MA','NH','RI','VT','NJ','NY','PA')) then 'Northeast' when (cs.state in ('IL','IN','MI',
'OH','WI','IA','KS','MN','MO','NE','ND','SD')) then 'Midwest' when (cs.state in ('DE','FL','GA','MD','NC','SC','VA','DC','WV','AL','KY','MS','TN','AR','LA','OK','TX')) t
hen 'South' when (cs.state in ('AZ','CO','ID','MT','NV','NM','UT','WY','AK','CA','HI','OR','WA')) then 'West' end) = r.'name') (cost=0.25 rows=1) (actual time=0.00501..0
.00597 rows=0.345 loops=4099)
                                -> Single-row index lookup on cs using PRIMARY (college_id = cm.college_id) (cost=0.25 rows=1) (actual time=0.0037..0.00375 rows=
1 loops=4099)
|

```

Index 3 (majors)

```
CREATE INDEX idx_majors ON majors(code, major_id);
```

Output:

```

| -> Limit: 15 row(s) (actual time=35.9..35.9 rows=15 loops=1)
    -> Sort: matched_major_count DESC, cs.'name' (actual time=35.9..35.9 rows=15 loops=1)
        -> Filter: ('count(distinct user_majors.'code') >= 1) (actual time=31.6..35.1 rows=557 loops=1)
        -> Stream results (actual time=31.6..35.1 rows=557 loops=1)
            -> Group aggregate: count(distinct user_majors.'code'), count(distinct user_majors.'code') (actual time=31.6..34.4 rows=557 loops=1)
            -> Sort: cs.college_id, cs.'name', cs.city, cs.state (actual time=31.6..32.4 rows=1415 loops=1)
            -> Stream results (cost=1093 rows=2405) (actual time=0.242..29 rows=1415 loops=1)
                -> Nested loop inner join (cost=1093 rows=2405) (actual time=0.236..27.7 rows=1415 loops=1)
                    -> Nested loop inner join (cost=251 rows=2405) (actual time=0.211..2.61 rows=4099 loops=1)
                        -> Nested loop inner join (cost=2.83 rows=3) (actual time=0.0893..0.113 rows=3 loops=1)
                            -> Nested loop inner join (LooseScan) (cost=1.78 rows=3) (actual time=0.071..0.086 rows=3 loops=1)
                                -> Remove duplicates from input sorted on idx_regions (cost=0.725 rows=1) (actual time=0.0537..0.0598 rows=1 lo
ops=1)
                                -> Covering index lookup on r using idx_regions (user_id = 4) (cost=0.725 rows=1) (actual time=0.0507..0.05
6 rows=1 loops=1)
                                -> Filter: (um.'code' is not null) (cost=1.05 rows=3) (actual time=0.0151..0.0229 rows=3 loops=1)
                                    -> Covering index lookup on um using idx_user_majors_user_id_code (user_id = 4) (cost=1.05 rows=3) (actual
time=0.0139..0.0207 rows=3 loops=1)
                                    -> Single-row covering index lookup on m using PRIMARY (major_id = um.'code') (cost=0.283 rows=1) (actual time=0.00
82..0.00823 rows=1 loops=3)
                                        -> Covering index lookup on cm using idx_college_majors (major_id = um.'code') (cost=29.2 rows=802) (actual time=0.0997
..0.683 rows=1366 loops=3)
                                        -> Filter: ((case when (cs.state in ('CT','ME','MA','NH','RI','VT','NJ','NY','PA')) then 'Northeast' when (cs.state in ('IL',
'IN','MI','OH','WI','IA','KS','MN','MO','NE','ND','SD')) then 'Midwest' when (cs.state in ('DE','FL','GA','MD','NC','SC','VA','DC','WV','AL','KY','MS','TN',
'AR','LA','OK','TX')) then 'South' when (cs.state in ('AZ','CO','ID','MT','NV','NM','UT','WY','AK','CA','HI','OR','WA')) then 'West' end) = r.'name') (cos
t=0.25 rows=1) (actual time=0.00583..0.00588 rows=0.345 loops=4099)
                                        -> Single-row index lookup on cs using PRIMARY (college_id = cm.college_id) (cost=0.25 rows=1) (actual time=0.00389..0
.00395 rows=1 loops=4099)
|

```

Analysis

We analyzed several indexing strategies to improve the performance of this query, focusing on join attributes and filtering conditions. The baseline execution plan already demonstrated relatively efficient performance (cost = 1086), with most joins utilizing existing primary or foreign key indexes. However, the query incurred significant overhead from duplicate

elimination, as indicated by the “Remove duplicate rows using temporary table” operation, which is required due to the `COUNT(DISTINCT um.code)` aggregation. We introduced a composite index on `regions(user_id, name)` to improve access patterns for both filtering and joining, but this resulted in little to no measurable improvement in cost. This is likely because MySQL was already using an index on `user_id`, and the small size of the filtered dataset (only a few rows per user) limited the benefit of further optimization.

Meanwhile, adding an index on majors actually slightly increased the cost to 1093. While additional indexes are generally expected to improve query performance, adding multiple indexes simultaneously in this case led to an increase in cost. This is because many of the indexed attributes were already efficiently accessed using existing primary or foreign key indexes, making the new indexes redundant. In particular, the composite index on `majors(code, major_id)` was poorly aligned with the query’s join condition on `major_id`, preventing effective utilization due to MySQL’s leftmost prefix rule. As a result, the optimizer may have selected less efficient execution paths. Furthermore, the main performance bottlenecks, which duplicate elimination from `COUNT(DISTINCT ...)` and row-by-row evaluation of the CASE expression, are not addressed by indexing, so the additional indexes provided no benefit in those areas. Instead, they introduced overhead and increased plan complexity, ultimately leading to higher estimated cost. This demonstrates that indiscriminately adding indexes can degrade performance, and that effective indexing requires careful alignment with query structure and access patterns.

In conclusion, the default index is the most effective to use, since it results in the lowest query cost and avoids unnecessary overhead.

Query 3

Baseline performance

```
| -> Limit: 15 row(s) (actual time=0.593..0.6 rows=15 loops=1)
      -> Sort: value_ratio DESC, avg_cost (actual time=0.592..0.597 rows=15 loops=1)
      -> Filter: ('count(distinct college_majors.major_id)' >= 1) (actual time=0.53..0.569 rows=15 loops=1)
      -> Stream results (actual time=0.528..0.566 rows=15 loops=1)
      -> Group aggregate: count(distinct college_majors.major_id), avg(colleges_dynamic.avg_cost_of_attendance), avg(colleges_dynamic.median_earnings_10yr), avg(colleges_dynamic.median_earnings_10yr), avg(colleges_dynamic.avg_cost_of_attendance), count(distinct college_majors.major_id) (actual time=0.522..0.546 rows=15 loops=1)
      -> Sort: cs.college_id, cs.name, cs.state, cd.year (actual time=0.505..0.508 rows=15 loops=1)
      -> Stream results (cost=645 rows=802) (actual time=0.182..0.464 rows=15 loops=1)
      -> Nested loop inner join (cost=645 rows=802) (actual time=0.175..0.44 rows=15 loops=1)
      -> Nested loop inner join (cost=364 rows=802) (actual time=0.0866..0.171 rows=15 loops=1)
      -> Nested loop inner join (cost=83.9 rows=802) (actual time=0.072..0.0803 rows=15 loops=1)
      -> Nested loop inner join (cost=1.2 rows=1) (actual time=0.0544..0.0559 rows=1 loops=1)
      -> Filter: (um.code is not null) (cost=0.85 rows=1) (actual time=0.0386..0.0399 rows=1 loops=1)
      -> Covering index lookup on um using idx_user_majors_user_id_code (user_id = 903) (cost=0.85 rows=1) (actual time=0.0373..0.0385 rows=1 loops=1)
      -> Single-row covering index lookup on m using PRIMARY (major_id = um.code) (cost=0.35 rows=1) (actual time=0.0146..0.0147 rows=1 loops=1)
      -> Covering index lookup on cm using idx_college_majors (major_id = um.code) (cost=82.7 rows=802) (actual time=0.017..0.0222 rows=15 loops=1)
      -> Single-row index lookup on cs using PRIMARY (college_id = cm.college_id) (cost=0.25 rows=1) (actual time=0.00572..0.00578 rows=1 loops=15)
      -> Filter: (cd.year = (select #2)) (cost=0.25 rows=1) (actual time=0.0162..0.0175 rows=1 loops=15)
      -> Index lookup on cd using PRIMARY (college_id = cm.college_id) (cost=0.25 rows=1) (actual time=0.00493..0.0061 rows=1 loops=15)
      -> Select #2 (subquery in condition; dependent)
      -> Aggregate: max(cd2.year) (cost=0.58 rows=1) (actual time=0.00925..0.00933 rows=1 loops=15)
      -> Covering index lookup on cd2 using PRIMARY (college_id = cd.college_id) (cost=0.35 rows=1) (actual time=0.0071..0.00837 rows=1 loops=15)
|
```

Index 1 (user_majors)

```
CREATE INDEX idx_user_majors_user_code ON user_majors(user_id, code);
```

Output:

```
| -> Limit: 15 row(s) (actual time=0.545..0.55 rows=15 loops=1)
      -> Sort: value_ratio DESC, avg_cost (actual time=0.544..0.547 rows=15 loops=1)
      -> Filter: ('count(distinct college_majors.major_id)' >= 1) (actual time=0.467..0.508 rows=15 loops=1)
      -> Stream results (actual time=0.466..0.505 rows=15 loops=1)
      -> Group aggregate: count(distinct college_majors.major_id), avg(colleges_dynamic.avg_cost_of_attendance), avg(colleges_dynamic.median_earnings_10yr), avg(colleges_dynamic.median_earnings_10yr), avg(colleges_dynamic.avg_cost_of_attendance), count(distinct college_majors.major_id) (actual time=0.46..0.485 rows=15 loops=1)
      -> Sort: cs.college_id, cs.name, cs.state, cd.year (actual time=0.443..0.447 rows=15 loops=1)
      -> Stream results (cost=645 rows=802) (actual time=0.119..0.404 rows=15 loops=1)
      -> Nested loop inner join (cost=645 rows=802) (actual time=0.112..0.38 rows=15 loops=1)
      -> Nested loop inner join (cost=364 rows=802) (actual time=0.0737..0.16 rows=15 loops=1)
      -> Nested loop inner join (cost=83.9 rows=802) (actual time=0.0603..0.0684 rows=15 loops=1)
      -> Nested loop inner join (cost=1.2 rows=1) (actual time=0.0447..0.0462 rows=1 loops=1)
      -> Filter: (um.code is not null) (cost=0.85 rows=1) (actual time=0.031..0.0322 rows=1 loops=1)
      -> Covering index lookup on um using idx_user_majors_user_id_code (user_id = 903) (cost=0.85 rows=1) (actual time=0.0298..0.0309 rows=1 loops=1)
      -> Single-row covering index lookup on m using PRIMARY (major_id = um.code) (cost=0.35 rows=1) (actual time=0.0122..0.0123 rows=1 loops=1)
      -> Covering index lookup on cm using idx_college_majors (major_id = um.code) (cost=82.7 rows=802) (actual time=0.015..0.0198 rows=15 loops=1)
      -> Single-row index lookup on cs using PRIMARY (college_id = cm.college_id) (cost=0.25 rows=1) (actual time=0.00571..0.00576 rows=1 loops=15)
      -> Filter: (cd.year = (select #2)) (cost=0.25 rows=1) (actual time=0.013..0.0143 rows=1 loops=15)
      -> Index lookup on cd using PRIMARY (college_id = cm.college_id) (cost=0.25 rows=1) (actual time=0.00485..0.00599 rows=1 loops=15)
      -> Select #2 (subquery in condition; dependent)
      -> Aggregate: max(cd2.year) (cost=0.58 rows=1) (actual time=0.0061..0.00615 rows=1 loops=15)
      -> Covering index lookup on cd2 using PRIMARY (college_id = cd.college_id) (cost=0.35 rows=1) (actual time=0.00413..0.00533 rows=1 loops=15)
|
```

Index 2 (subquery attributes)

```
CREATE INDEX idx_cd_college_year_median
ON colleges_dynamic(college_id, year, median_earnings_10yr,
avg_cost_of_attendance);
```

Output:

```
| -> Limit: 15 row(s) (actual time=0.545..0.55 rows=15 loops=1)
    -> Sort: value_ratio DESC, avg_cost (actual time=0.544..0.547 rows=15 loops=1)
        -> Filter: ('count(distinct college_majors.major_id)' >= 1) (actual time=0.467..0.508 rows=15 loops=1)
            -> Stream results (actual time=0.466..0.505 rows=15 loops=1)
                -> Group aggregate: count(distinct college_majors.major_id), avg(colleges_dynamic.avg_cost_of_attendance), avg(colleges_dynamic.median_earnings_10yr), avg(colleges_dynamic.median_earnings_10yr), avg(colleges_dynamic.avg_cost_of_attendance), count(distinct college_majors.major_id) (actual time=0.46..0.485 rows=15 loops=1)
                    -> Sort: cs.college_id, cs.'name', cs.state, cd.'year' (actual time=0.443..0.447 rows=15 loops=1)
                        -> Stream results (cost=645 rows=802) (actual time=0.119..0.404 rows=15 loops=1)
                            -> Nested loop inner join (cost=645 rows=802) (actual time=0.112..0.38 rows=15 loops=1)
                                -> Nested loop inner join (cost=364 rows=802) (actual time=0.0737..0.16 rows=15 loops=1)
                                    -> Nested loop inner join (cost=83.9 rows=802) (actual time=0.0603..0.0684 rows=15 loops=1)
                                        -> Nested loop inner join (cost=1.2 rows=1) (actual time=0.0447..0.0462 rows=1 loops=1)
                                            -> Filter: (um.'code' is not null) (cost=0.85 rows=1) (actual time=0.031..0.0322 rows=1 loops=1)
                                                -> Covering index lookup on um using idx_user_majors_user_id_code (user_id = 903) (cost=0.85 rows=1) (actual time=0.0298..0.0309 rows=1 loops=1)
                                                    -> Single-row covering index lookup on m using PRIMARY (major_id = um.'code') (cost=0.35 rows=1) (actual time=0.0122..0.0123 rows=1 loops=1)
                                                        -> Covering index lookup on cm using idx_college_majors (major_id = um.'code') (cost=82.7 rows=802) (actual time=0.015..0.0198 rows=15 loops=1)
                                                            -> Single-row index lookup on cs using PRIMARY (college_id = cm.college_id) (cost=0.25 rows=1) (actual time=0.00571..0.00576 rows=1 loops=15)
                                                                -> Filter: (cd.'year' = (select #2)) (cost=0.25 rows=1) (actual time=0.013..0.0143 rows=1 loops=15)
                                                                    -> Index lookup on cd using PRIMARY (college_id = cm.college_id) (cost=0.25 rows=1) (actual time=0.00485..0.00599 rows=1 loops=15)
                                                                        -> Select #2 (subquery in condition; dependent)
                                                                            -> Aggregate: max(cd2.'year') (cost=0.58 rows=1) (actual time=0.0061..0.00615 rows=1 loops=15)
                                                                                -> Covering index lookup on cd2 using PRIMARY (college_id = cd.college_id) (cost=0.35 rows=1) (actual time=0.00413..0.00533 rows=1 loops=15)
|
```

Index 3 (majors)

```
CREATE INDEX idx_m_major_id_code
ON majors(major_id, code);
```

Output:

```
| -> Limit: 15 row(s) (actual time=0.545..0.55 rows=15 loops=1)
    -> Sort: value_ratio DESC, avg_cost (actual time=0.544..0.547 rows=15 loops=1)
        -> Filter: ('count(distinct college_majors.major_id)' >= 1) (actual time=0.467..0.508 rows=15 loops=1)
            -> Stream results (actual time=0.466..0.505 rows=15 loops=1)
                -> Group aggregate: count(distinct college_majors.major_id), avg(colleges_dynamic.avg_cost_of_attendance), avg(colleges_dynamic.median_earnings_10yr), avg(colleges_dynamic.median_earnings_10yr), avg(colleges_dynamic.avg_cost_of_attendance), count(distinct college_majors.major_id) (actual time=0.46..0.485 rows=15 loops=1)
                    -> Sort: cs.college_id, cs.'name', cs.state, cd.'year' (actual time=0.443..0.447 rows=15 loops=1)
                        -> Stream results (cost=645 rows=802) (actual time=0.119..0.404 rows=15 loops=1)
                            -> Nested loop inner join (cost=645 rows=802) (actual time=0.112..0.38 rows=15 loops=1)
                                -> Nested loop inner join (cost=364 rows=802) (actual time=0.0737..0.16 rows=15 loops=1)
                                    -> Nested loop inner join (cost=83.9 rows=802) (actual time=0.0603..0.0684 rows=15 loops=1)
                                        -> Nested loop inner join (cost=1.2 rows=1) (actual time=0.0447..0.0462 rows=1 loops=1)
                                            -> Filter: (um.'code' is not null) (cost=0.85 rows=1) (actual time=0.031..0.0322 rows=1 loops=1)
                                                -> Covering index lookup on um using idx_user_majors_user_id_code (user_id = 903) (cost=0.85 rows=1) (actual time=0.0298..0.0309 rows=1 loops=1)
                                                    -> Single-row covering index lookup on m using PRIMARY (major_id = um.'code') (cost=0.35 rows=1) (actual time=0.0122..0.0123 rows=1 loops=1)
                                                        -> Covering index lookup on cm using idx_college_majors (major_id = um.'code') (cost=82.7 rows=802) (actual time=0.015..0.0198 rows=15 loops=1)
                                                            -> Single-row index lookup on cs using PRIMARY (college_id = cm.college_id) (cost=0.25 rows=1) (actual time=0.00571..0.00576 rows=1 loops=15)
                                                                -> Filter: (cd.'year' = (select #2)) (cost=0.25 rows=1) (actual time=0.013..0.0143 rows=1 loops=15)
                                                                    -> Index lookup on cd using PRIMARY (college_id = cm.college_id) (cost=0.25 rows=1) (actual time=0.00485..0.00599 rows=1 loops=15)
                                                                        -> Select #2 (subquery in condition; dependent)
                                                                            -> Aggregate: max(cd2.'year') (cost=0.58 rows=1) (actual time=0.0061..0.00615 rows=1 loops=15)
                                                                                -> Covering index lookup on cd2 using PRIMARY (college_id = cd.college_id) (cost=0.35 rows=1) (actual time=0.00413..0.00533 rows=1 loops=15)
|
```

We evaluated several indexing strategies for this query, focusing on optimizing join conditions and the correlated subquery used to retrieve the most recent year per college. However, unlike previous queries, indexing had no measurable impact on performance. The execution plan remained unchanged across all configurations, with an already low cost (645) and minimal execution time. This is primarily due to the highly selective filter on `user_id = 903`, which reduced the dataset to a single row early in the execution plan. As a result, subsequent joins produced only a small number of rows (approximately 15), allowing MySQL to efficiently process the query using existing primary and foreign key indexes. Additionally, the correlated subquery was executed only 15 times, making it inexpensive compared to similar patterns in earlier queries. The execution plan shows that all joins were already performed using index lookups, leaving no opportunity for further improvement through additional indexing. The remaining operations, such as `COUNT(DISTINCT ...)`, `AVG(...)`, and sorting, are computational in nature and cannot be optimized through indexing. This demonstrates that when a query is highly selective and already uses efficient access paths, additional indexes provide no benefit and may be redundant.

In conclusion, the default index is the most effective to use, since adding custom indexes did not result in a significant increase in performance.